Getting Started with Data Version Control (DVC)

Why Versioning Data?

Working with Machine learning and data analysis often involve working with large datasets that change over time. Unlike code, managing datasets has unique problems:

- Changes to datasets are not easily trackable.
- Collaboration becomes difficult without a system to synchronize datasets across team members.
- Difficulties in Reproducibility when the dataset used for a specific model version is lost or modified.

Versioning data solves these issues by allowing:

- **Traceability:** Know exactly which dataset version corresponds to a specific model.
- **Reproducibility:** Recreate experiments with the same datasets used earlier.
- **Collaboration:** Share data efficiently within teams.

What is Data Version Control (DVC)?

DVC is an open-source tool designed to bring version control capabilities to data, making it work alongside traditional version control systems like Git.

- It tracks datasets and machine learning models in a Git-like fashion.
- DVC stores actual data in external storage (local, cloud, or remote), while saving lightweight pointers in the Git repository.
- It makes it easy for data sharing, reproducibility, and efficient storage by handling large files outside the repository.

How Does DVC Work?

DVC works in conjunction with Git to manage data:

- **Versioning:** Tracks changes in datasets by storing metadata in `.dvc` files.
- **Storage:** Data is stored in external storage backends (e.g., local directories, S3, Google Drive).
- **Pointers:** Git stores lightweight pointers (e.g., `.dvc` files) instead of the actual large datasets.
- **Automation:** DVC automates pulling and pushing data from/to the storage backend.

Using DVC

This guide walks you through setting up DVC in a project and managing your datasets.

1. Install DVC

First, we will need Git, as DVC relies on it. Then, download DVC from https://dvc.org/#get-started-dvc.

Verify the installation:

```
dvc --version
```

2. Initialize DVC in Your Project

DVC needs to be initialized in your project directory. Start by creating a new project:

```
mkdir my-dvc-project
cd my-dvc-project
git init
dvc init
```

This creates a `.dvc` directory for configuration and adds it to your Git repository.

3. Add a Dataset to DVC

Place your dataset in the project directory (e.g., `data/raw_dataset.csv`) and add it to DVC:

```
dvc add data/raw_dataset.csv
```

This creates a `.dvc` file (e.g., `raw_dataset.csv.dvc`) that acts as a pointer to the data file. DVC then prompts you to add the generated `.dvc` and `.gitignore` files to Git. Stage and commit the `.dvc` file to Git:

```
git add data/raw_dataset.csv.dvc data/.gitignore
git commit -m "Add raw dataset to DVC"
```

If you're working with a remote repo, and you do a `git push`, you'll notice the data file will not be pushed. This is because DVC handles data storage separately from the Git repository.

4. Configure Remote Storage

Set up remote storage to back up and share datasets.

Example 1: Amazon S3

```
dvc remote add -d myremote s3://mybucket/path
```

Push the dataset to the remote storage:

```
dvc push
```

Example 2: Google Drive

Install Google Drive Dependencies

```
pip install "dvc[gdrive]"
```

Configure Google Drive

1. Create a folder in your Google Drive (e.g., `DVC_Storage`).
2. Obtain the folder ID from the URL. For example:

   ```
   https://drive.google.com/drive/folders/<FOLDER_ID>?
   <NOT_IMPORTANT_PARAMS>
   ```

3. Add the Google Drive remote to your DVC configuration:

   ```
   dvc remote add -d myremote gdrive://<FOLDER_ID>
   ```

4. Push the dataset to the remote storage:

   ```
   dvc push
   ```

Example 3: Azure Blob Storage (SAS Token Authentication)

Azure Blob Storage uses a **storage account** and a **blob container**, rather than an S3 bucket. For this project:

- **Storage account:** `wut2026`
- **Blob container:** `dvc`
- **DVC remote name:** `myazure`
- **Container URL in DVC:** `azure://dvc`

1. Install DVC with Azure Support

```
python -m pip install -U "dvc[azure]"
```

2. Configure the Azure Blob Storage Remote

```
dvc remote add -d myazure azure://dvc
```

If the remote `myazure` already exists, update its URL instead:

```
dvc remote modify myazure url azure://dvc
```

Set the Azure storage account name. This is required even when authenticating with a SAS token:

```
dvc remote modify myazure account_name wut2026
```

DVC identifies the storage location from the account name and the `azure://dvc` container URL. The full `https://wut2026.blob.core.windows.net/dvc` URL is not required in `dvc remote add`.

3. Set the SAS Token Locally

The following token is valid until **2026-12-12 at 10:59:11 UTC**:

```
SAS_TOKEN='sp=racwdl&st=2026-10-15T01:44:11Z&se=2026-12-12T10:59:11Z'
SAS_TOKEN="${SAS_TOKEN}&spr=https&sv=2026-02-06&sr=c"
SAS_TOKEN="${SAS_TOKEN}&sig=mN2DNZjMYXCB%2B%2BqqZBhkvDFz0ZrvAnzqtRtI4YnO06s
%3D"
dvc remote modify --local myazure sas_token "$SAS_TOKEN"
unset SAS_TOKEN
```

The `--local` option stores the SAS token in `.dvc/config.local`, which DVC normally excludes from Git. **Do not commit this file or the SAS token to your repository.**

> **Security:** This Markdown guide itself contains a live SAS token. Keep this document private and do not commit it to a public or shared repository. For a shareable copy, replace the token with a placeholder. If the token is exposed, revoke or rotate it in Azure.

4. Check the Remote Configuration

```
dvc remote list
dvc config remote.myazure.account_name
```

The account name should be `wut2026`. To upload DVC-tracked data:

```
dvc push
```

If DVC reports `Authentication to Azure Blob Storage requires either account_name or connection_string`, make sure the account name is configured:

```
dvc remote modify myazure account_name wut2026
```

5. Share the Project

Share the repository by pushing it to a Git remote:

```
git remote add origin <repository_url>
git push -u origin main
```

Other collaborators can clone the repository and pull the dataset:

```
git clone <repository_url>
dvc pull
```

6. Track Changes to the Dataset

If the dataset changes, DVC tracks the updates. For example, after modifying `data/raw_dataset.csv`:

```
dvc add data/raw_dataset.csv
git add data/.gitignore raw_dataset.csv.dvc
git commit -m "Update dataset"
dvc push
```

7. Hands-On Experiment: Compare Dataset Versions

This experiment lets you **edit a local CSV file, save two versions with DVC and Git, upload both to Azure Blob Storage, and switch between them**. Use a small demo dataset so your real data is not affected.

Before starting, make sure DVC is initialized, the Azure remote `myazure` is configured, and Git has no unrelated uncommitted changes (`git status`). Run these commands from the project root.

1. Create and save version 1

Create a CSV file containing two records:

```
mkdir -p data
printf 'id,score\n1,80\n2,90\n' > data/dvc_demo.csv
cat data/dvc_demo.csv
```

Track it with DVC, commit the pointer file with Git, tag the version, and upload the dataset to Azure:

```
dvc add data/dvc_demo.csv
git add data/dvc_demo.csv.dvc data/.gitignore
git commit -m "Add demo dataset version 1"
git tag dvc-demo-v1
dvc push
```

2. Edit the local dataset and save version 2

Add a third record (you can also edit `data/dvc_demo.csv` in your text editor):

```
printf '3,95\n' >> data/dvc_demo.csv
cat data/dvc_demo.csv
dvc status
```

DVC should detect that the tracked file has changed. Save and upload the new version:

```
dvc add data/dvc_demo.csv
git add data/dvc_demo.csv.dvc
git commit -m "Add demo dataset version 2"
git tag dvc-demo-v2
dvc push
```

3. Inspect the two versions

Use Git to see the history of the DVC pointer file and compare its metadata between versions:

```
git log --oneline -- data/dvc_demo.csv.dvc
git diff dvc-demo-v1 dvc-demo-v2 -- data/dvc_demo.csv.dvc
```

You will see the pointer's hash change, **not the CSV rows**. Git stores the `.dvc` metadata; DVC stores the actual contents in its cache and Azure remote.

4. Switch back to version 1, then forward to version 2

The commands below use Git to select a committed `.dvc` pointer, then `dvc pull` to retrieve the matching dataset. Run them in the **same terminal session** so `ORIGINAL_BRANCH` is preserved:

```
ORIGINAL_BRANCH="$(git branch --show-current)"

# View version 1
git switch --detach dvc-demo-v1
```

```
dvc pull
cat data/dvc_demo.csv
```

Expected contents for version 1:

```
id,score
1,80
2,90
```

Now switch to version 2:

```
git switch --detach dvc-demo-v2
dvc pull
cat data/dvc_demo.csv
```

Expected contents for version 2:

```
id,score
1,80
2,90
3,95
```

5. Return to your original branch

```
git switch "$ORIGINAL_BRANCH"
dvc pull
```

The dataset is restored to the version recorded by your current branch. **Key concept:** `git switch` changes which dataset version is referenced, while `dvc pull` materializes the corresponding data. `dvc push` uploads dataset content; `git commit` records the version pointer. Do not make uncommitted edits while switching versions, since restoring data can overwrite them.

8. Clean Up Large Files

DVC optimizes storage by removing unnecessary large files:

```
dvc gc
```

This removes unused files from local and remote storage.